SkillForge Python Mastery

120 Most Common & Important Python Interview Questions with Answers

SET 1: BEGINNER LEVEL (0-2 Years) - 60 Questions

58 Questions - Interview Ready

Q1. What is Python? Why is it popular?

Ans: High-level, interpreted, dynamically-typed, garbage-collected language by Guido van Rossum. Popular for simple syntax, readability, vast libraries (Django, Flask, Pandas, TensorFlow), community, and use in Web, Data Science, AI, Automation. Zen of Python: import this.

Q2. Python 2 vs Python 3 differences?

Ans: Python2 EOL 2020. Python3: print is function, unicode default (str is unicode), 5/2=2.5 not 2, xrange removed, dict.items() returns view, more strict. Always use Python3.

Q3. What is PEP 8?

Ans: Official style guide. 4 spaces indent, snake_case for functions/vars, PascalCase for classes, line length 79/88, imports at top, 2 blank lines between classes. Enforced by flake8, black, isort. Improves readability and collaboration.

Q4. Mutable vs Immutable?

Ans: Mutable can change after creation: list, dict, set, bytearray. Immutable cannot: int, float, str, tuple, frozenset. Immutable are hashable, can be dict keys. Mutable default arg bug originates from mutability.

Q5. Difference between List, Tuple, Set, Dict?

Ans: List: ordered, mutable, duplicates allowed []. Tuple: ordered, immutable, (). Set: unordered, mutable, unique {}. Dict: key-value, ordered since 3.7, {k:v}. Use List for sequence, Tuple for fixed record, Set for membership, Dict for mapping.

Q6. What is List Comprehension?

Ans: Concise way to create lists, faster than loop. [expr for item in iterable if cond]. Eg: [x*x for x in range(10) if x%2==0]. Also dict {k:v for...} and set {x for...}. Avoid over-nesting.

Q7. Explain *args and **kwargs?

Ans: *args collects extra positional args as tuple. **kwargs collects extra keyword args as dict. Order: positional, *args, keyword-only, **kwargs. Use for flexible APIs and decorators.

Q8. What is __init__ and self?

Ans: __init__ is initializer/constructor, called after __new__ creates object. self is explicit reference to current instance, first param of instance methods. Python requires explicit self unlike this in Java.

Q9. Difference between is and == ?

Ans: == checks value equality via __eq__. is checks identity (same id in memory). Use is for None: if x is None. Small ints -5 to 256 interned, so is may be True but don't rely.

Q10. Deep copy vs Shallow copy?

Ans: Shallow copy copies outer object but inner nested objects share reference (copy.copy(), list[:]). Deep copy recursively copies all (copy.deepcopy()). Deep copy expensive, uses memo dict to handle cycles.

Q11. What is GIL?

Ans: Global Interpreter Lock - mutex in CPython allowing only one thread to execute Python bytecode at a time. So CPU-bound threads don't parallelize. IO-bound threads release GIL during IO. For CPU parallelism use multiprocessing or Python 3.13 free-threaded build.

Q12. How does Python memory management work?

Ans: Private heap, pymalloc for <512 bytes objects, reference counting primary + cyclic GC for cycles. Every object has refcount. When 0, deallocated. gc module handles circular refs in 3 generations. Use tracemalloc, sys.getrefcount().

Q13. What are Decorators?

Ans: Higher-order functions that modify behavior of another function. Takes function, returns wrapper. Syntax @decorator. Common: @property, @staticmethod, @lru_cache, @wraps. Essential for logging, auth, caching.

Q14. Iterators vs Generators?

Ans: Iterator implements __iter__ and __next__. Generator is easy iterator using yield, lazy, memory efficient, stateful. def gen(): yield 1. Generator expression: (x*x for x in range(10)). Generators are one-time use.

Q15. What is lambda?

Ans: Anonymous small function: lambda args: expression. Single expression only. Use with sorted, map, filter. Eg: sorted(data, key=lambda x: x[1]). Avoid complex logic, use def for readability.

Q16. Explain map(), filter(), reduce()?

Ans: map(func, iter): applies func to each -> map object. filter(func, iter): keeps where True. reduce(func, iter) from functools cumulatively reduces. Prefer list comprehensions now for readability: [func(x) for x in iter if cond].

Q17. How exception handling works?

Ans: try: risky, except SpecificError as e: handle, else: runs if no exception, finally: always runs for cleanup. Catch specific, not bare except. Create custom by inheriting Exception. Use raise, raise from.

Q18. What is with statement?

Ans: with ensures resource cleanup via context manager __enter__/__exit__. with open('f') as f: auto closes even on error. Create custom via class or @contextmanager decorator with yield.

Q19. What is slicing?

Ans: Extracting part of sequence: seq[start:stop:step], start inclusive stop exclusive. Negative from end. a[::-1] reverse. Returns copy, doesn't raise IndexError if out of range. Powerful for string/list manipulation.

Q20. Docstring vs Comments?

Ans: Comment # for dev notes, ignored. Docstring triple-quoted string first statement in module/class/function, stored in __doc__, used by help(). Follow PEP 257 Google/NumPy style.

Q21. What are pass, break, continue?

Ans: pass: no-op placeholder for empty block. break: exit loop completely. continue: skip current iteration to next. Used in loops.

Q22. global vs local vs nonlocal?

Ans: local: inside function. global: declare to modify module-level var inside function. nonlocal: inside nested function to modify enclosing function var (Python3). Avoid global, use params/return.

Q23. Modules vs Packages?

Ans: Module is single .py file. Package is directory with __init__.py containing modules. Namespace packages without __init__.py since 3.3. Import via import, from...import. Search via sys.path.

Q24. pip vs conda vs venv?

Ans: pip: official installer from PyPI. conda: cross-language package+env manager from Anaconda. venv/virtualenv: isolated environments to avoid conflict. Always use: python -m venv env and pip freeze > requirements.txt

Q25. What does if __name__ == '__main__' do?

Ans: When file run directly __name__='__main__'. When imported __name__=module name. Guard runs only when executed directly, not on import. Best practice for scripts.

Q26. What is mutable default argument bug?

Ans: def func(a=[]): a.append(1) default list created once at definition, shared across calls causing growth. Fix: def func(a=None): if a is None: a=[]. Classic trap.

Q27. How to handle large files?

Ans: Don't read() all. Use for line in open(): iterator, or chunk read(8192), or generators. For CSV use csv module streaming, for JSON ijson. Use mmap for memory-mapped.

Q28. __str__ vs __repr__?

Ans: __str__ user-friendly (str(), print). __repr__ unambiguous dev representation (repr(), debugger). If __str__ missing uses __repr__. Implement __repr__ ideally eval(repr(obj)) works.

Q29. @staticmethod vs @classmethod?

Ans: Instance: self, access instance/class. @classmethod: cls first arg, access class state, alternative constructors. @staticmethod: no self/cls, utility inside class namespace.

Q30. How import system works?

Ans: Checks sys.modules cache, finds spec via sys.meta_path finders, loads via loader, executes, caches. Order: builtins, sys.path. Circular imports cause partial init.

Q31. What is dynamic typing?

Ans: Type checked at runtime, not compile time. Variable can hold any type and change: x=5 then x='hi'. Strongly typed (no implicit coercion like 1+'1' fails). Check via isinstance(). Use type hints for safety.

Q32. What is None?

Ans: None is singleton representing absence of value, like null. NoneType. Falsy. Should check with 'is None' not ==. Function returns None if no return. Mutable vs None default.

Q33. How to check type?

Ans: Use isinstance(obj, type) preferred over type(obj)==type for inheritance support. type() gives exact type. Use typing for hints. Avoid type checking often - duck typing preferred.

Q34. range vs xrange?

Ans: Python2: range returns list, xrange returns iterator lazy. Python3: range is like xrange - lazy sequence, memory efficient, supports len, slicing. Use list(range()) if need list.

Q35. How to reverse list/string?

Ans: List reverse: list.reverse() in-place, or list[::-1] returns copy, or reversed(list) iterator. String: s[::-1] since strings immutable. Also ''.join(reversed(s)).

Q36. How to remove duplicates from list?

Ans: Use set: list(set(lst)) but loses order. Preserve order: list(dict.fromkeys(lst)) since 3.7 dict ordered. Or manual loop with seen set. For unhashable, use loop.

Q37. How to sort dict by value/key?

Ans: Sorted returns list of keys: sorted(d) by key, sorted(d, key=d.get) by value, sorted(d.items(), key=lambda x:x[1]). To get dict: dict(sorted(...)). Use operator.itemgetter for speed.

Q38. What is unpacking? * operator?

Ans: * unpacks iterable, ** unpacks dict. a,b = [1,2], a,*rest,b = [1,2,3,4], [*list1, *list2] merging, {**d1, **d2} merging. In function calls: func(*args).

Q39. enumerate and zip?

Ans: enumerate(iterable, start=0) returns (index,value) pairs, avoids manual counter. zip(iter1, iter2) pairs elements until shortest, use zip_longest for longest. Unzip: zip(*pairs). Both lazy iterators.

Q40. append vs extend?

Ans: append(x): adds x as single element, even if x is list -> nested. extend(iterable): iterates and adds each element individually. Eg: [1,2].append([3]) => [1,2,[3]], extend([3]) => [1,2,3].

Q41. File open modes?

Ans: r read, w write truncate, a append, x exclusive create, b binary, t text default, + update. Eg: 'rb', 'w+', 'a+'. Use with open for safety. Default encoding utf-8.

Q42. How to handle JSON?

Ans: json module: json.loads(str) parse, json.dumps(obj) serialize. Handles dict/list/str/int/float/None. For custom objects use default param. Difference from pickle: JSON human-readable, cross-language, secure.

Q43. What is pickling?

Ans: Serializing Python object to byte stream via pickle module. pickle.dumps/loads. Can pickle almost any Python object, preserves type. Not secure - never unpickle untrusted data. Use JSON for interop.

Q44. What is __slots__?

Ans: __slots__ = ['x','y'] in class restricts attributes to listed, no __dict__, saves memory, faster attribute access. Useful for many instances. Tradeoff: no dynamic attributes, multiple inheritance complexity.

Q45. del vs remove vs pop?

Ans: del: statement removes variable, list element by index, slice, or dict key, no return. remove(value): list method removes first matching value, ValueError if not found. pop(index): removes and returns value, dict pop(key).

Q46. What is assert?

Ans: assert condition, message checks invariant during development. If False raises AssertionError. Disabled with -O optimization. Use for debugging, not for validating user input (use if + raise).

Q47. Recursion limit?

Ans: Python limits recursion depth default 1000 via sys.getrecursionlimit() to avoid stack overflow. Can increase via sys.setrecursionlimit(). Python has no tail recursion optimization. Use iteration for deep recursion.

Q48. Why indentation important?

Ans: Python uses indentation to define blocks, not braces. 4 spaces per PEP8. Mixing tabs/spaces causes IndentationError/TabError. Enforces readability. Use editor that shows whitespace.

Q49. LEGB rule?

Ans: Scope resolution order: Local, Enclosing, Global, Built-in. Python searches L->E->G->B for name. Enclosing is for nested functions. Use global/nonlocal to modify outer scopes.

Q50. How to generate random numbers?

Ans: random module: random.random() 0-1 float, randint(a,b) inclusive, choice(list), shuffle(list) in-place, sample(population,k) without replacement. For crypto secure use secrets module: secrets.token_hex().

Q51. What is ternary operator?

Ans: Conditional expression: value_if_true if condition else value_if_false. Eg: x = 'even' if n%2==0 else 'odd'. Concise but avoid nesting for readability.

Q52. input() vs raw_input()?

Ans: Python2: raw_input() returns string, input() evals input (dangerous). Python3: input() is Python2's raw_input() returning string, raw_input removed. Always convert: int(input()).

Q53. How to handle command line args?

Ans: sys.argv list first element script name. Better use argparse for flags, help, type validation. Example: parser=argparse.ArgumentParser(); parser.add_argument('--count', type=int). Also click library for CLI apps.

Q54. What is bytecode .pyc?

Ans: .pyc files are compiled bytecode in __pycache__/ folder, created on import to speed up loading, not for execution speed. .pyo optimized. Bytecode executed by Python VM. .pyc is Python version specific.

Q55. What is id() and type()?

Ans: id(obj) returns unique identity integer (memory address in CPython). type(obj) returns type object. Use id for identity tracking, but use 'is' for identity comparison.

Q56. How to copy file in Python?

Ans: shutil module: shutil.copy(src,dst) copies file and permission, copy2 preserves metadata, copyfileobj for file objects, copytree for directories. For large files use copyfileobj with buffer.

Q57. What is requirements.txt?

Ans: Text file listing pinned dependencies: package==version. Created via pip freeze > requirements.txt. Installed via pip install -r requirements.txt. Best practice: use pip-tools or poetry for dependency management.

Q58. What is virtual environment and why?

Ans: Isolated Python installation with own interpreter and packages, avoids version conflicts between projects. Create via python -m venv myenv, activate via source myenv/bin/activate. Essential for every project.

SET 2: PROFESSIONAL LEVEL (10+ Years) - 60 Questions

53 Questions - Interview Ready

Q59. Explain Python memory management in depth?

Ans: Private heap, pymalloc for <512B, reference counting Py_INCREF/DECREF, when 0 freed. Cyclic GC 3 generations detects isolated cycles via gc.collect(). pymalloc arenas. Memory not always returned to OS. Use tracemalloc, gc, objgraph, sys.getsizeof.

Q60. GIL impact and true parallelism?

Ans: GIL allows one thread to execute Python bytecode, so CPU-bound threads don't scale. IO-bound releases GIL during IO so benefit. True parallelism: multiprocessing (separate memory, pickle overhead), ProcessPoolExecutor, C extensions releasing GIL (numpy), Python 3.13 free-threaded build PEP703.

Q61. Decorator with arguments and functools.wraps?

Ans: Decorator with args = 3 levels: outer takes args, middle takes func, inner wrapper *args/**kwargs. Use @functools.wraps(func) to preserve __name__, __doc__, __module__, __wrapped__. Otherwise debugging/stack trace broken.

Q62. What are Metaclasses? When to use?

Ans: Metaclass is class of class, defines how class is created. type is default metaclass. Override __new__(mcs,name,bases,dict) to customize. Use: ORM field validation, Singleton registry, API auto-registration. 99% time use class decorator instead, metaclass overkill.

Q63. Explain MRO and C3 Linearization?

Ans: MRO is order Python looks for method. C3 linearization preserves local precedence and monotonicity. Diamond: A, B(A), C(A), D(B,C) => D->B->C->A->object. View via Class.__mro__. super() follows MRO not just parent, crucial for cooperative multiple inheritance.

Q64. asyncio vs threading vs multiprocessing?

Ans: Threading: IO-bound, shared memory, low overhead but GIL limits CPU. Multiprocessing: CPU-bound, true parallelism, high memory, pickle needed. asyncio: single thread cooperative multitasking via event loop, 10k connections, async/await non-blocking. FastAPI uses asyncio for IO heavy APIs.

Q65. How does dict work internally?

Ans: Hash table open addressing. Hash key -> index, collision via probing. Python 3.6+ uses compact representation: indices array + dense entries array to preserve insertion order and save memory. Average O(1) get/set. Hash randomized via SipHash for DOS protection.

Q66. Time complexity of List, Dict, Set?

Ans: List: index O1, append O1 amortized, insert/delete On, search On, sort O(n log n). Dict/Set: get/set/delete O1 avg, On worst. Membership: list On vs set O1. Use set for membership test. bisect for sorted list.

Q67. Explain Descriptor Protocol?

Ans: Descriptor customizes attribute access. Implements __get__(self,obj,objtype), __set__, __delete__. Data descriptor defines __set__ has precedence over instance dict. Used for @property, @classmethod,

ORM fields. property() is descriptor.

Q68. Closures and late binding pitfall?

Ans: Closure captures variable from enclosing scope, not value. Bug: [lambda: i for i in range(3)] all return 2 because i looked up at call time. Fix: lambda i=i: i captures via default arg. Use functools.partial.

Q69. Thread-safe Singleton in Python?

Ans: Module-level singleton simplest (module imported once). Or metaclass with Lock: class SingletonMeta(type): _instances={}, _lock=Lock(); def __call__(cls,*a,**kw): if cls not in _instances: with _lock: if cls not...; _instances[cls]=super().__call__(*a,**kw). Or use @lru_cache on __new__.

Q70. __new__ vs __init__?

Ans: __new__(cls) creates and returns instance, static method, first. __init__(self) initializes already created instance, returns None. __new__ used for immutable types (str,int,tuple) or Singleton, controlling creation. Usually override __init__ only.

Q71. How GC handles circular references?

Ans: Refcount alone can't collect cycles (a refs b, b refs a). Generational GC tracks container objects in 3 generations, periodically traverses, finds isolated groups with only internal refs, frees. gc.disable() for perf but risk leak. gc.garbage holds uncollectable with __del__.

Q72. What is monkey patching? Risks?

Ans: Dynamically modifying class/module at runtime: requests.get = my_mock. Useful for testing/mocking with unittest.mock.patch. Risks: breaks encapsulation, hard to debug, thread-unsafe, fragile to version changes. Avoid in prod, use DI.

Q73. Explain Python type hinting and modern typing?

Ans: Type hints PEP484, not enforced at runtime, checked by mypy/pyright. List[int], Dict[str,Any], Optional[X], Union, X|Y in 3.10+. TypeVar, Generic, Protocol structural typing (duck), TypedDict, Literal, Final, Annotated. Improves IDE and maintainability.

Q74. How to make code thread-safe?

Ans: GIL doesn't guarantee safety for compound ops (check-then-act). Use threading.Lock, RLock, Semaphore, Event. Use queue.Queue thread-safe. Protect shared data with with lock:. For multiprocessing use multiprocessing.Lock. Avoid global state.

Q75. What is context manager deep dive?

Ans: __enter__ returns resource, __exit__(exc_type,exc_val,tb) handles exception, return True suppresses. Create via class or @contextmanager with yield. Multiple: with A() as a, B() as b:. Use for transactions, timing, suppress, locking.

Q76. How to optimize Python performance?

Ans: 1) Algorithm 2) Use built-ins (list comp in C) 3) Profile cProfile, line_profiler, py-spy 4) PyPy for loops 5) Numpy vectorization 6) Cython/Numba for hot loops 7) Multiprocessing 8) LRU cache 9) Local var binding in loops. Python 3.11 25% faster.

Q77. New features in Python 3.11/3.12/3.13?

Ans: 3.11: 10-60% faster, Exception groups except*, tomllib. 3.12: f-string improvements (multi-line, quotes), type alias 'type', override decorator, perf. 3.13: free-threaded no-GIL experimental (PEP703), JIT experimental, improved REPL, removal dead batteries. match/case since 3.10.

Q78. How to handle memory leaks?

Ans: Tools: tracemalloc to track alloc, gc.get_objects(), objgraph growth, memory_profiler, pympler. Common leaks: global lists growing, __del__ with cycles, ThreadLocal not cleared, C extensions not freeing, lru_cache unbounded. Use weakref.WeakValueDictionary for caches.

Q79. Explain asyncio event loop, coroutine, Task, Future?

Ans: Event loop single thread schedules coroutines. coroutine async def needs await. Future low-level placeholder. Task is Future subclass wrapping coroutine execution. asyncio.create_task() schedules. await asyncio.gather() concurrent. Never block loop with sync IO - use to_thread.

Q80. Difference between generator, iterator, iterable?

Ans: Iterable has __iter__ returning iterator (list,str). Iterator has __next__ and __iter__, stateful one-time. Generator function with yield returns iterator, lazy. All generators are iterators, all iterators are iterables. Use iter() to get iterator.

Q81. How to design scalable Python API?

Ans: FastAPI/Starlette async, Pydantic validation, DI. Layers: routers, services, repositories. Uvicorn workers 2*CPU. Caching Redis, rate limiting, DB pooling asyncpg. Logging structlog, metrics prometheus, tracing OTEL. Follow 12-factor.

Q82. Import lock and circular import fix?

Ans: Import lock ensures thread-safe import. Circular: A imports B, B imports A -> partially initialized. Fixes: import inside function, refactor common to C, use TYPE_CHECKING for hints, import module not from import symbol, use importlib.

Q83. Explain property, setter, deleter?

Ans: @property makes method act like attribute computed on access. Setter via @prop.setter, deleter via @prop.deleter. Useful for validation, lazy computation, read-only. Property uses descriptor protocol internally.

Q84. How to implement LRU Cache from scratch?

Ans: Use OrderedDict or dict+DLL for O1 get/set. Python has functools.lru_cache decorator and OrderedDict. Custom: hash map + doubly linked list, maintain size, evict least recently used. Use weakref for memory sensitive.

Q85. Shallow vs deep copy implementation?

Ans: Shallow: copy.copy() - new outer, inner refs same - uses __copy__. Deep: copy.deepcopy() - recursively copies, memo dict handles cycles - uses __deepcopy__. Deep slow, custom __deepcopy__ for perf.

Q86. Handle large file/dataset without OOM?

Ans: Streaming: for line in open(), generators yield, chunk read, csv.reader streaming, ijson for JSON, fetchmany for DB, memory-mapped mmap. For >RAM use Dask, PySpark, Vaex. Use generators pipeline.

Q87. Explain walrus operator := ?

Ans: Assignment expression PEP572 Python 3.8, assigns and returns value in same expression. Use to avoid duplicate calls: if (n := len(data)) > 10: print(n). Useful in while, comprehensions, if conditions. Avoid overuse.

Q88. Dataclasses vs attrs vs namedtuple vs Pydantic?

Ans: namedtuple: immutable tuple with named fields, lightweight. dataclass: Python 3.7 @dataclass generates __init__, __repr__, __eq__, mutable, type hints, fast. attrs: more features than dataclass. Pydantic: dataclass + validation, parsing, JSON schema, used in FastAPI.

Q89. What is __slots__ memory optimization?

Ans: __slots__ = ['x','y'] restricts attributes, no __dict__, saves ~50% memory per instance, faster attribute access. Useful for millions of instances. Tradeoff: no dynamic attributes, weakref needs include '__weakref__', multiple inheritance complex.

Q90. What is weakref? Use cases?

Ans: Weak reference doesn't increase refcount, doesn't prevent GC. Use WeakValueDictionary, WeakSet, WeakKeyDictionary for caches that don't prevent collection, observer pattern, circular refs. weakref.ref(obj, callback) callback on finalization.

Q91. How multiprocessing handles memory? Copy-on-write?

Ans: Multiprocessing spawns new process via fork (Unix) copy-on-write - memory shared until written, efficient. But on Windows spawns new interpreter. Data passed via pickle. Shared memory via Value, Array, Manager, or shared_memory module Python 3.8+. Avoid large pickle overhead.

Q92. How to write C extension and Cython?

Ans: C extension: write C code using Python C API, compile via setuptools. Cython: superset of Python that compiles to C, add type hints cdef int, 10-100x speedup for loops. Use pyx file, cythonize. Numba @jit also compiles via LLVM for numeric code.

Q93. Explain import hooks and meta path?

Ans: sys.meta_path list of finders that locate module spec. sys.path_hooks. Custom finder implements find_spec(). Loader implements create_module(), exec_module(). Use for loading from DB, network, encrypted. Used by importlib.

Q94. How to implement plugin system?

Ans: Via entry points (setuptools), importlib.metadata, or dynamic import of modules in plugin folder. Use pluggy library (used by pytest). Or simple: discover .py files, import via importlib.import_module, register via decorator. Use Protocol for plugin interface.

Q95. Handle logging in multi-process?

Ans: Logging not process-safe by default file handler interleaves. Use QueueHandler + QueueListener (Python 3.2+) to send logs via multiprocessing.Queue to listener process that writes. Or use structlog, or log to separate files per process, or use external aggregator like ELK.

Q96. How to handle secrets and env vars?

Ans: Never hardcode secrets. Use environment variables, dotenv (.env file) with python-dotenv, or secret managers: AWS Secrets Manager, Vault. Use pydantic-settings for validation. For Docker use secrets. Avoid committing .env to git.

Q97. Explain concurrent.futures?

Ans: High-level API for async execution: ThreadPoolExecutor for IO-bound, ProcessPoolExecutor for CPU-bound. Submit returns Future. Use executor.submit(func) or executor.map(). as_completed() for results as they finish. Easier than manual threading/multiprocessing.

Q98. Explain deadlock and how to avoid in Python?

Ans: Deadlock when two threads wait for each other's lock. Conditions: mutual exclusion, hold&wait;, no preemption, circular wait. Avoid by lock ordering, using with statement, tryLock with timeout, using RLock for re-entrancy, avoiding nested locks, using queue for communication.

Q99. Race condition example in Python?

Ans: Race when threads access shared data without sync. Example: counter +=1 not atomic: read, increment, write. Two threads may lose update. Despite GIL, compound ops not atomic. Fix with Lock. Use threading.Lock() and with lock: counter+=1. Use queue for thread-safe communication.

Q100. Hash randomization and security?

Ans: Python randomizes hash seed via PYTHONHASHSEED to prevent DoS via hash collision attack (attacker crafts keys with same hash to make dict O(n^2)). Since Python 3.3 enabled by default via SipHash. Affects reproducibility - set PYTHONHASHSEED=0 for debugging.

Q101. How to secure pickle?

Ans: Pickle can execute arbitrary code on unpickle - never unpickle untrusted data. Alternative: JSON, MessagePack. If must use pickle, use hmac signing, or restrict with Unpickler overriding find_class to allow only safe classes. Use pickle protocol 4+ for efficiency.

Q102. How to handle large JSON streaming?

Ans: Standard json loads entire file into memory. For large files use ijson (iterative parser), jsonlines, or parse chunks. For writing stream via json.dump iteratively. For huge JSON use databases or convert to line-delimited JSON (JSONL).

Q103. How to implement rate limiter?

Ans: Token bucket or fixed window algorithm. Use time.monotonic(), threading.Lock. For distributed use Redis with INCR and EXPIRE, or redis-cell. In Python implement decorator with deque of timestamps. Libraries: limits, ratelimit, slowapi for FastAPI.

Q104. How to implement circuit breaker?

Ans: Pattern to prevent cascading failures. States: Closed (normal), Open (fail fast), Half-Open (test). After N failures, open for timeout. Use pybreaker library or custom via decorator counting failures, tracking time. Important for microservices calling external APIs.

Q105. How to test async code?

Ans: Use pytest-asyncio: mark test with @pytest.mark.asyncio, use async def test_(). Use AsyncMock from unittest.mock for mocking async functions. Use httpx AsyncClient for testing FastAPI. Use asyncio.run() for

simple scripts. Avoid calling asyncio.run() inside event loop.

Q106. Python GC tuning?

Ans: gc.set_threshold() controls generational collection frequency (700,10,10 default). Lower threshold more frequent GC. gc.set_debug(gc.DEBUG_STATS) to monitor. Disable GC during critical perf section gc.disable() then enable. Use gc.freeze() for 3.7+ to not scan immutable objects.

Q107. Explain GIL removal in 3.13 free-threaded?

Ans: PEP703 introduces free-threaded build --disable-gil, allows true parallel threads, reference counting becomes atomic, biased reference counting. Requires changes to C extensions. Still experimental, many extensions not compatible. Future may make Python truly multi-threaded for CPU-bound without multiprocessing.

Q108. How to handle deadlock with asyncio?

Ans: Asyncio deadlock via await circular dependency or blocking call inside event loop. Avoid blocking IO (use async version), avoid await inside lock held long, use asyncio.Lock not threading.Lock in async code, use asyncio.wait_for with timeout to detect deadlock.

Q109. What is dataclass vs Pydantic performance?

Ans: Dataclass fast, minimal overhead, no validation. Pydantic v1 slower due to validation, v2 written in Rust 5-50x faster. Use dataclass for internal models, Pydantic for API validation. attrs similar to dataclass but more features. Choose based on need for validation.

Q110. How to implement observer pattern in Python?

Ans: Use list of callbacks, or weakref.WeakSet for observers to avoid leaks. Or use signals library like blinker, PyDispatcher. Example: class Subject: def attach(self, obs): self._obs.append(weakref.ref(obs)); def notify(self): for o in self._obs: o().update(). Or use property descriptor to auto notify.

Q111. How to handle Python packaging and dependency management?

Ans: Modern: pyproject.toml per PEP 621, use poetry or pdm or hatch or uv for deps, build with build module. Old: setup.py. Use twine to upload to PyPI. Use pip-tools to pin. Use virtual env. Use wheel for distribution. Follow semantic versioning.

Interview Tips from SkillForge

For Beginner: Master fundamentals, write code on paper, know time complexity, practice list/dict/set operations.
For Professional: Expect deep dive into GIL, asyncio, memory, MRO, metaclasses, descriptors, profiling, system design.
Preparation: Build 2-3 projects with FastAPI/Django, read CPython source, know PEPs.